

Deep Learning

Session 2

Agenda:

1	Deeper Neural Networks and Hidden Layers
2	Overfitting and Regularization Techniques
3	Optimization Techniques

Deeper Neural Networks and Hidden Layers

Deeper Neural Networks and Hidden Layers

What is a Deep Neural Network (DNN)?

- A **Deep Neural Network (DNN)** is simply a neural network with more than one hidden layer between the input and output layers. These hidden layers enable the network to learn more complex patterns and features from the data. The addition of more layers allows the network to model hierarchical patterns, making DNNs powerful for tasks like image classification, speech recognition, and natural language processing.

Deeper Neural Networks and Hidden Layers

What is a Deep Neural Network (DNN)?

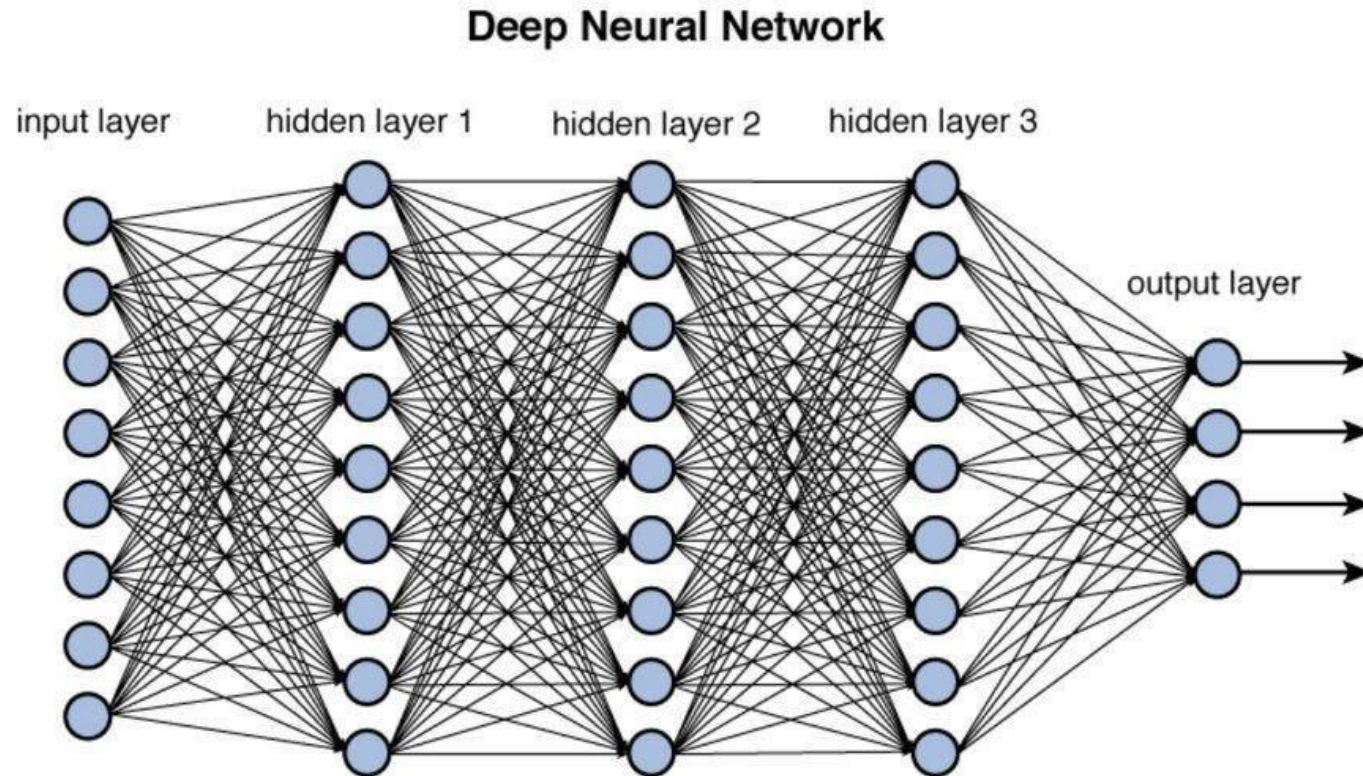


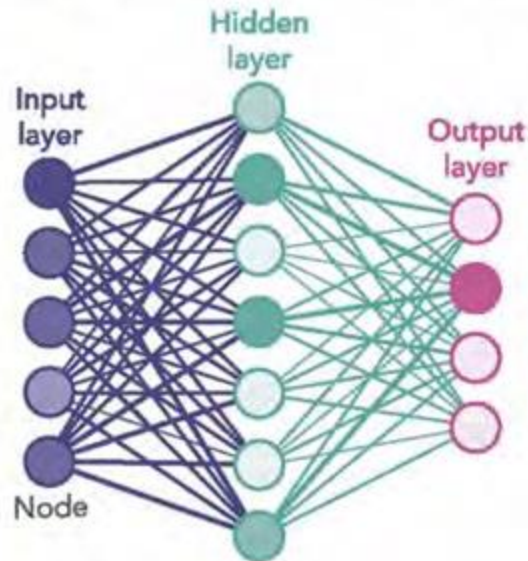
Figure 12.2 Deep network architecture with multiple layers.

Deeper Neural Networks and Hidden Layers

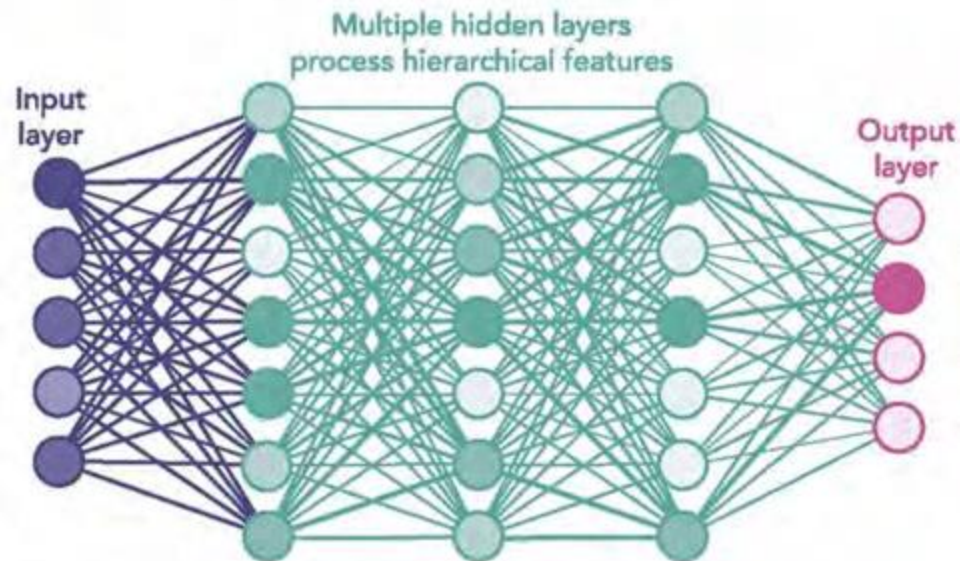
Why Add More Hidden Layers?

- **Simple Patterns in Shallow Networks:** A shallow neural network (with only one or two hidden layers) can only learn relatively simple patterns. For example, it may be able to differentiate between basic shapes in images.

SHALLOW NEURAL NETWORK



DEEP NEURAL NETWORK



Deeper Neural Networks and Hidden Layers

Why Add More Hidden Layers?

- **Complex Patterns in Deep Networks:** Adding more hidden layers enables the network to learn complex hierarchical representations. Each layer builds on features learned in previous layers :
 - The **first layer** may detect edges or basic features.
 - The **second layer** may detect shapes and contours.
 - The **deeper layers** may detect more complex features like objects or faces.

The deeper the network, the more complex and abstract the learned features become.

Deeper Neural Networks and Hidden Layers

How Data Passes Through a Deep Neural Network

- In a **deep neural network**, data passes through multiple layers of neurons. Each layer performs the following :

1. Weighted Sum

- Each neuron receives inputs from the previous layer and calculates a weighted sum.

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

Deeper Neural Networks and Hidden Layers

How Data Passes Through a Deep Neural Network

- In a **deep neural network**, data passes through multiple layers of neurons. Each layer performs the following :

2. Activation Function

- The weighted sum is passed through an activation function (e.g., ReLU, Sigmoid, Tanh) to introduce non-linearity.

Deeper Neural Networks and Hidden Layers

How Data Passes Through a Deep Neural Network

- In a **deep neural network**, data passes through multiple layers of neurons. Each layer performs the following :

3. Output of Layer

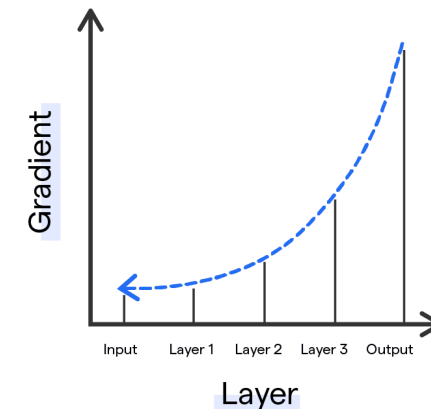
- The result is passed to the next layer as input, and this continues until the output layer.

Deeper Neural Networks and Hidden Layers

Vanishing Gradient Problem

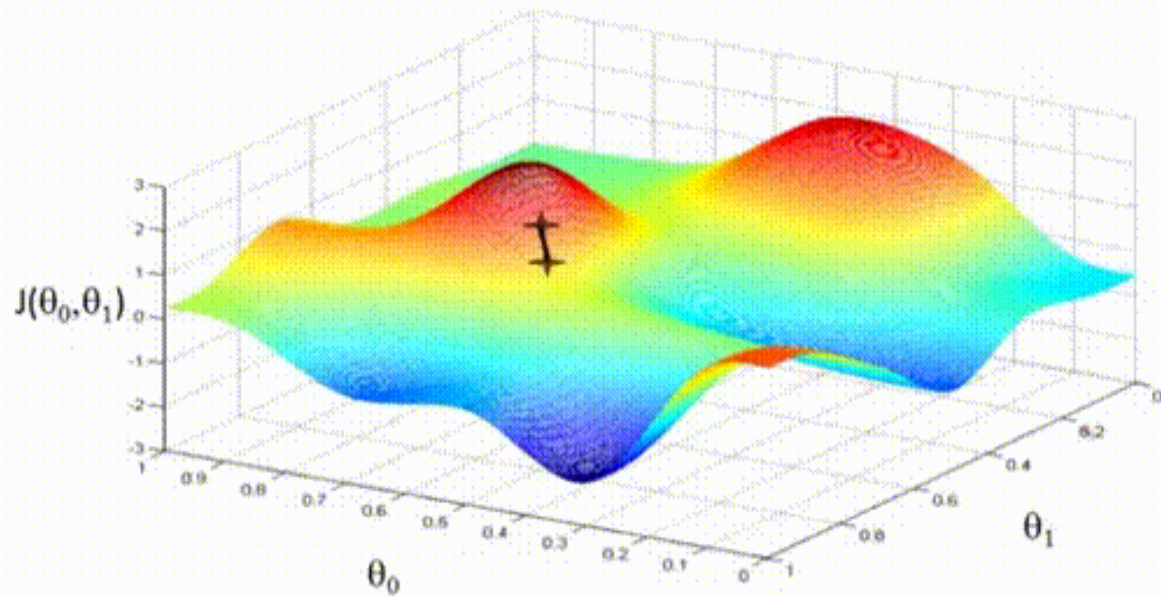
- The **Vanishing Gradient Problem** occurs during the training of deep neural networks, especially when using certain activation functions. It happens when the gradients of the loss function become very small (close to zero) as they are propagated back through the layers. As a result, the earlier layers in the network receive extremely small updates during training, making it difficult for the model to learn effectively in those layers.

Vanishing Gradient Problem



Deeper Neural Networks and Hidden Layers

Vanishing Gradient Problem

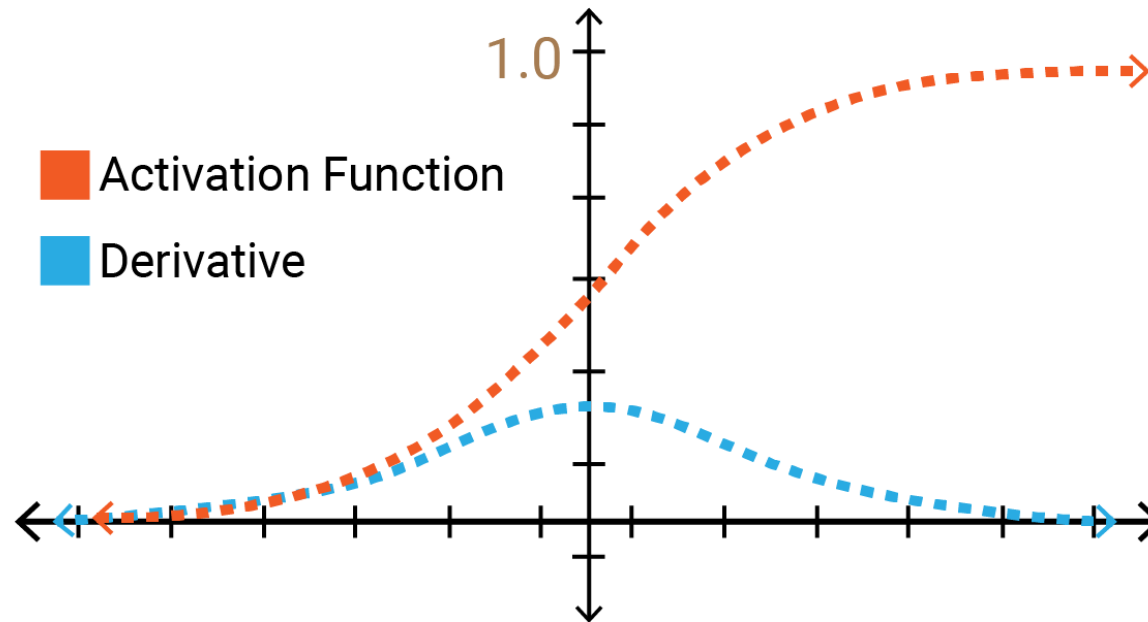


Andrew Ng

Deeper Neural Networks and Hidden Layers

How Activation Functions Affect the Vanishing Gradient Problem ?

- Different activation functions have varying effects on the vanishing gradient problem. Let's break down how common activation functions like **Sigmoid**, **Tanh**, and **ReLU** handle this issue.



Deeper Neural Networks and Hidden Layers

Sigmoid Activation Function

What is Sigmoid?

- The **sigmoid function** maps any input to a value between 0 and 1, and is defined as :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where z is the input (usually the weighted sum of inputs in a neuron).

Deeper Neural Networks and Hidden Layers

How Does Sigmoid Cause Vanishing Gradients?

- **Output Saturation:** The sigmoid function **saturates** at both ends of its range. When the input z is large and positive, the output is close to 1. When z is large and negative, the output is close to 0.
- **Gradient:** The derivative of the sigmoid function is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

This derivative is very small when z is large in magnitude (either positive or negative), meaning that the gradient for neurons in these regions becomes very small.

- For inputs far from 0, the gradient of sigmoid becomes very close to 0, leading to **vanishing gradients** during backpropagation.

Deeper Neural Networks and Hidden Layers

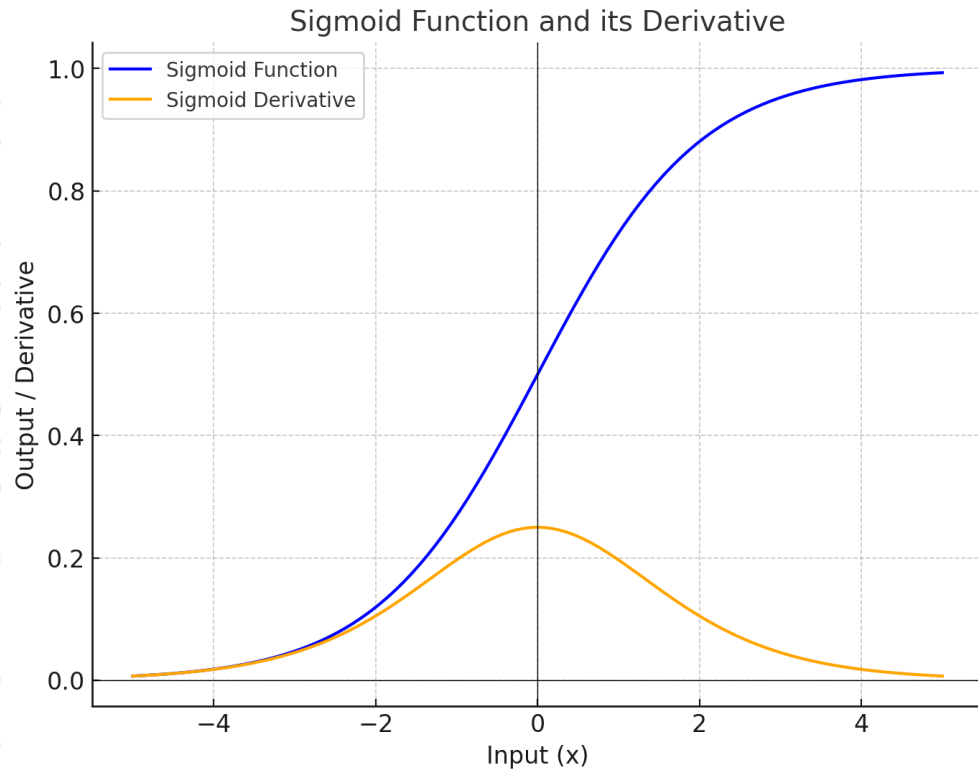
How Does Sigmoid Cause Vanishing Gradients?

Effect on Deep Networks

- In deep networks, layers closer to the input receive much smaller gradient updates, slowing down the training process significantly or even stopping learning altogether.

Deeper Neural Networks and Hidden Layers

How Does Sigmoid Cause Vanishing Gradients?



- The plot shows the **Sigmoid Function** (blue) and its **Derivative** (orange).
- The **gradient (derivative)** becomes very small when input values are far from 0 (both negative and positive).
- For large positive or negative input values, the derivative approaches zero.
- This behavior leads to the **vanishing gradient problem**, where gradients become too small during backpropagation, slowing down learning in deep neural networks.

Deeper Neural Networks and Hidden Layers

Tanh Activation Function

What is Tanh?

- The Tanh (hyperbolic tangent) function maps any input to a value between -1 and 1. It is defined as :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Deeper Neural Networks and Hidden Layers

How Does Tanh Handle Vanishing Gradients?

- **Output Range:** Like the sigmoid function, **tanh** can suffer from the vanishing gradient problem, but it does better than sigmoid in some cases because its output is centered around zero (-1 to 1), which helps to maintain a mean of zero for activations, allowing for faster convergence.
- **Gradient:** The derivative of the tanh function is :

$$\tanh'(z) = 1 - \tanh(z)^2$$

Like sigmoid, the gradient becomes very small for inputs with large magnitudes, resulting in the same issue of vanishing gradients. However, because its output range is [-1, 1] instead of [0, 1], it tends to perform slightly better in deeper networks.

Deeper Neural Networks and Hidden Layers

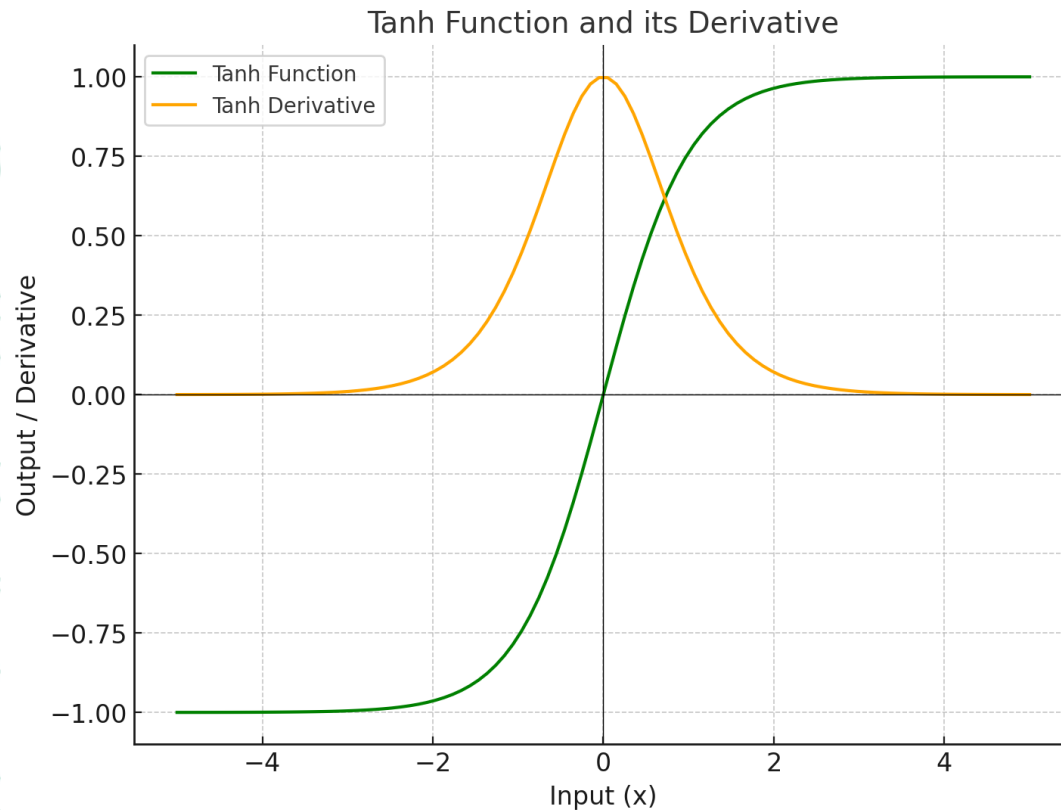
How Does Tanh Handle Vanishing Gradients?

Effect on Deep Networks

- While tanh is an improvement over sigmoid due to its zero-centered output, it still suffers from vanishing gradients for deep networks. The further from 0 the inputs are, the smaller the gradients, and the slower the training becomes for earlier layers.

Deeper Neural Networks and Hidden Layers

How Does Tanh Handle Vanishing Gradients?



- **Tanh Function** ranges between -1 and 1, which helps maintain a mean of zero for activations.
- The **Derivative of Tanh** becomes very small for inputs with large magnitudes, leading to **vanishing gradients** for large positive or negative inputs, similar to the Sigmoid function. However, since its output is centered around zero, it tends to perform better than Sigmoid in deeper networks

Deeper Neural Networks and Hidden Layers

ReLU (Rectified Linear Unit) Activation Function

What is ReLU?

- The **ReLU** activation function is defined as :

$$\text{ReLU}(z) = \max(0, z)$$

ReLU is a piecewise linear function that returns the input value if it is positive, and 0 if the input is negative.

Deeper Neural Networks and Hidden Layers

How Does ReLU Handle Vanishing Gradients?

- **No Saturation for Positive Inputs:** The key difference with ReLU is that it **does not saturate** for positive values. The gradient of ReLU is :

$$\frac{d}{dz}\text{ReLU}(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

This means that for positive inputs, the gradient remains 1, preventing the vanishing gradient problem and allowing for faster learning.

Deeper Neural Networks and Hidden Layers

Advantages of ReLU

- **Efficient Gradient Flow:** Because ReLU has a gradient of 1 for positive inputs, the gradient does not shrink as it propagates through the network, enabling more effective weight updates.
- **Sparsity:** ReLU sets negative values to 0, which introduces sparsity in the network. Sparse representations can make the network more efficient and can also help prevent overfitting.

Deeper Neural Networks and Hidden Layers

Potential Issue: Dying ReLU Problem

- While ReLU solves the vanishing gradient problem, it has its own issue called the **Dying ReLU Problem**. If a neuron consistently receives negative inputs, its output will be 0, and its gradient will be 0. Once this happens, the neuron may stop learning entirely.
- To address this, variants of ReLU like **Leaky ReLU** or **Parametric ReLU** introduce a small negative slope for negative inputs, preventing neurons from "dying."

Deeper Neural Networks and Hidden Layers

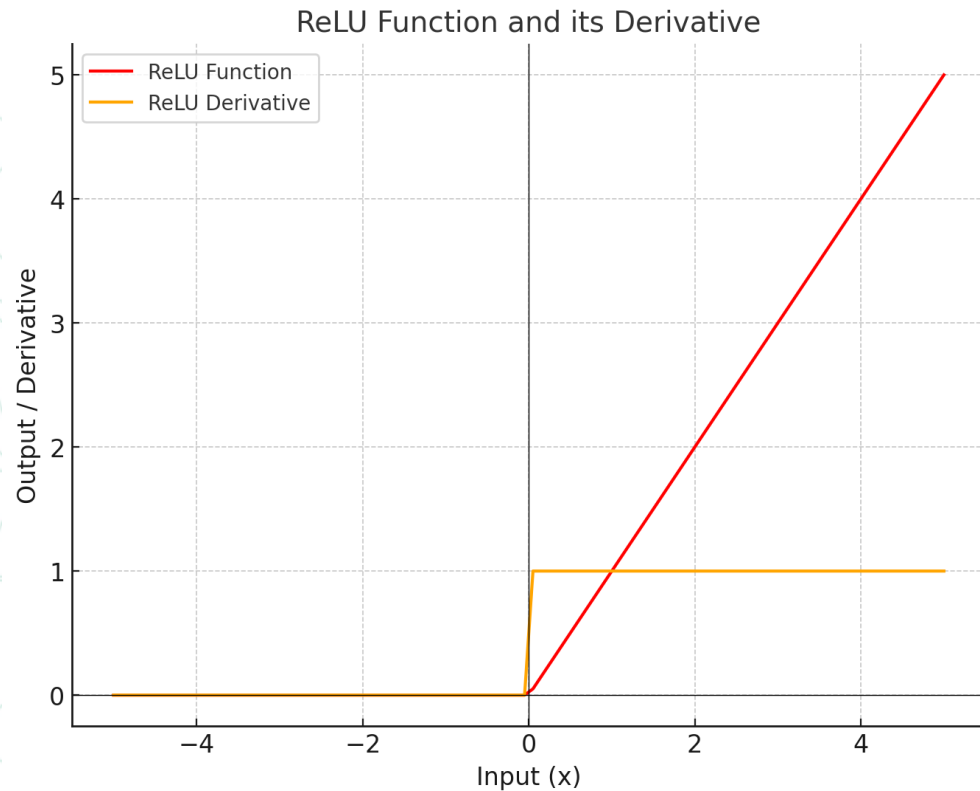
How Does ReLU Handle Vanishing Gradients?

Effect on Deep Networks

- ReLU has become the most commonly used activation function in deep networks because it **avoids the vanishing gradient problem**. It allows gradients to flow efficiently through the network, leading to faster and more reliable training, especially for deep architectures.

Deeper Neural Networks and Hidden Layers

How Does ReLU Handle Vanishing Gradients?



- **ReLU Function** does not saturate for positive inputs, meaning it continues to grow linearly as input values increase.
- The **Derivative of ReLU** is 1 for positive inputs and 0 for negative inputs. This prevents the vanishing gradient problem for positive values, allowing for faster learning in deep networks.

Overfitting and Regularization Techniques

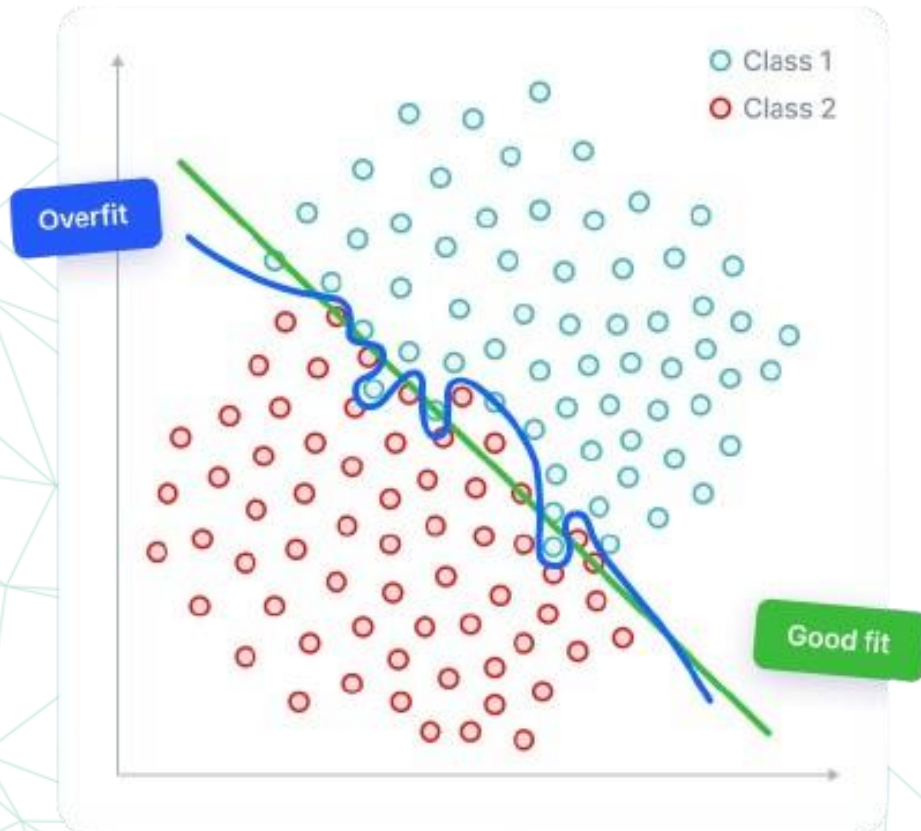
Overfitting and Regularization Techniques

What is Overfitting?

- **Overfitting** occurs when a neural network learns not only the underlying patterns in the training data but also the noise and outliers. As a result, the network performs very well on the training data but fails to generalize to new, unseen data (test or validation data).
- In simple terms, overfitting happens when the network is too complex (e.g., too many parameters) for the available training data, leading it to memorize the data rather than generalize it.

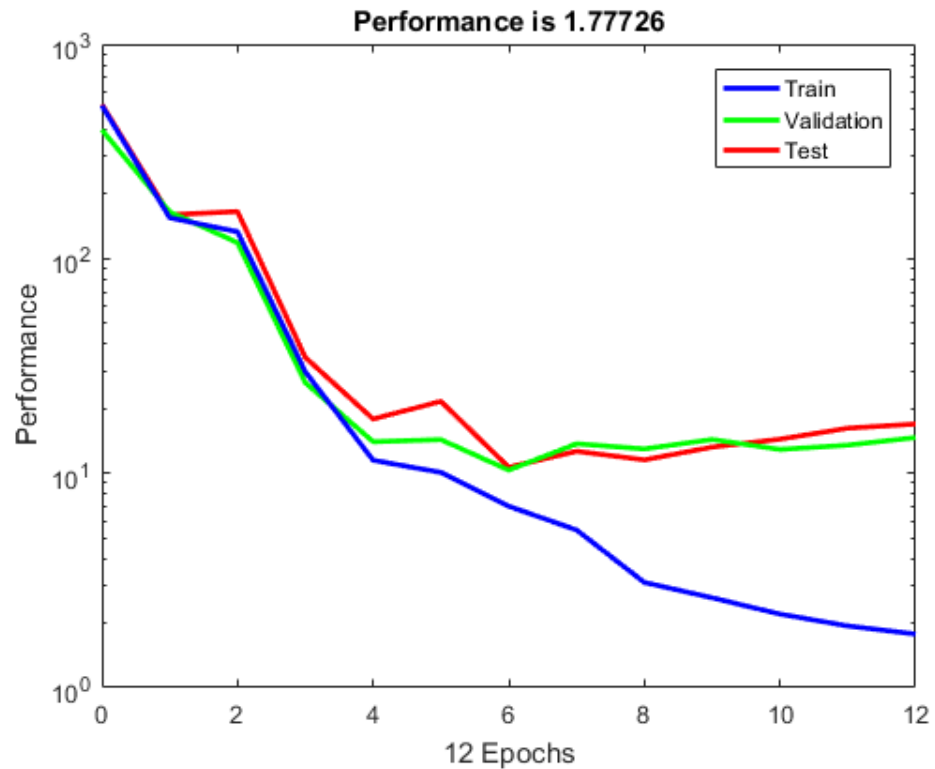
Overfitting and Regularization Techniques

What is Overfitting?



Overfitting and Regularization Techniques

Signs of Overfitting

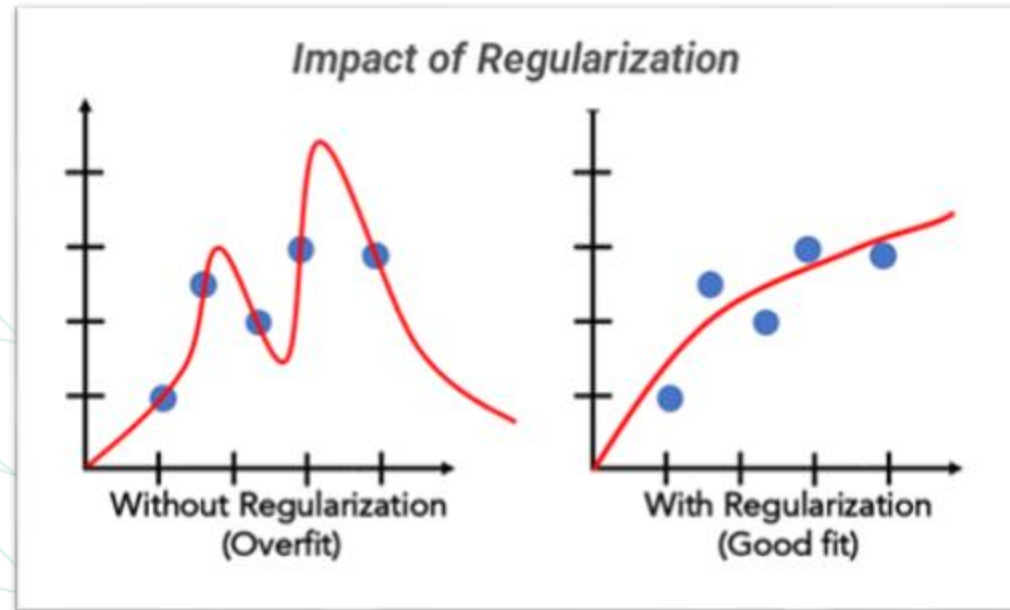


- **Low training error, but high validation/test error.**
- The network performs very well on the training data but poorly on unseen data.

Overfitting and Regularization Techniques

Regularization Techniques to Combat Overfitting

- To prevent overfitting, we can use several **regularization techniques**. These techniques prevent the model from being too complex and help it generalize better to new data.



Overfitting and Regularization Techniques

1. L2 Regularization (Weight Decay)

How it works ?

- L2 regularization adds a penalty to the loss function based on the sum of the squared weights.

Why it helps ?

- This discourages the network from assigning very high values to weights, effectively reducing the model complexity and preventing overfitting.

Overfitting and Regularization Techniques

1. L2 Regularization (Weight Decay)

Loss Function with L2 Regularization

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum w^2$$

Where :

- L_{new} is the regularized loss.
- L_{original} is the original loss function (e.g., cross-entropy or mean squared error).
- λ is the regularization parameter that controls the strength of the regularization.

Overfitting and Regularization Techniques

2. L1 Regularization

How it works ?

- L1 regularization adds the sum of the absolute values of the weights to the loss function.

Why it helps ?

- It can lead to sparse models, where some weights become zero, effectively simplifying the model by selecting important features and eliminating unnecessary ones.

Overfitting and Regularization Techniques

2. L1 Regularization

Loss Function with L1 Regularization

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum |w|$$

Overfitting and Regularization Techniques

3. Dropout

How it works ?

- Dropout randomly "drops" (sets to zero) a fraction of neurons during each forward pass of training. This prevents neurons from relying too much on each other and forces the network to learn more robust representations.

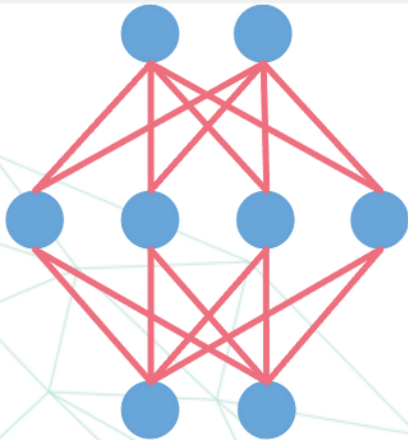
Why it helps ?

- Dropout introduces redundancy, which makes the network more robust to noise and less likely to overfit the training data.
- During training, a dropout rate (e.g., 0.5) is set, and during each training iteration, random neurons are deactivated with that probability.

Overfitting and Regularization Techniques

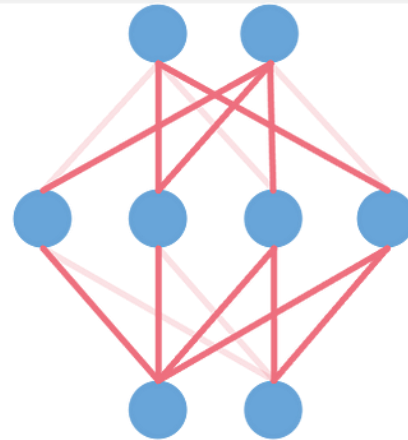
3. Dropout

Original Network

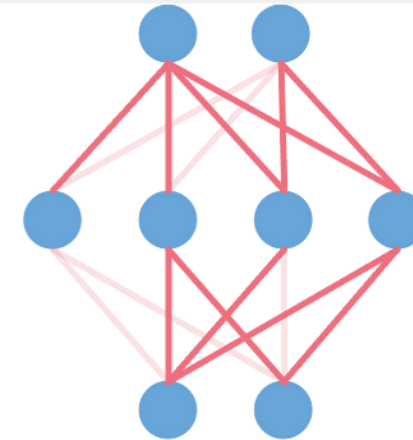


Connections = 16

Dropout-ratio = 30%



Active = 11 (70%)



Active = 11 (70%)

Overfitting and Regularization Techniques

4. Early Stopping

How it works ?

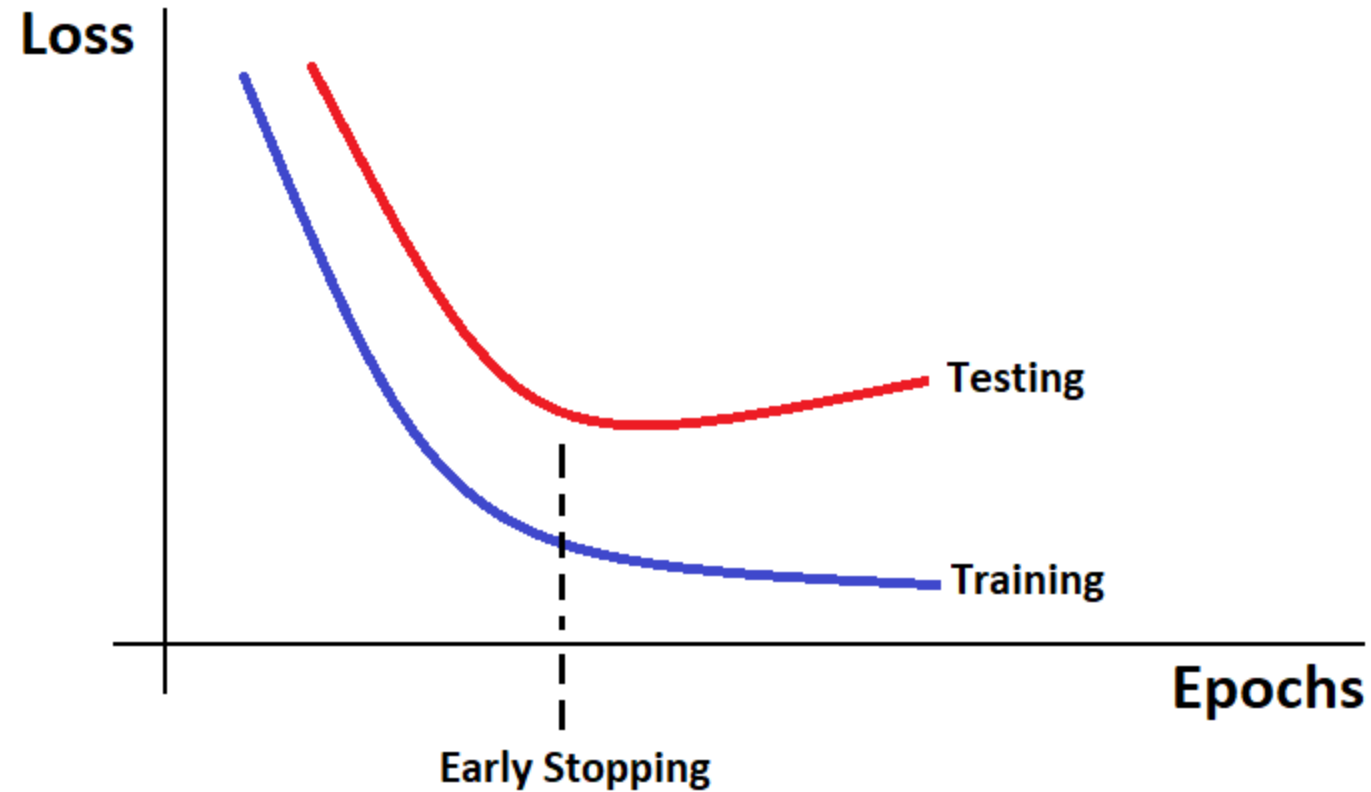
- Early stopping is a simple yet effective technique where training is stopped when the validation error starts to increase, even if the training error continues to decrease.

Why it helps ?

- This prevents the model from continuing to learn noise and overfit the training data.
- Typically, the validation loss is monitored, and when it starts increasing for several epochs, training is stopped.

Overfitting and Regularization Techniques

4. Early Stopping



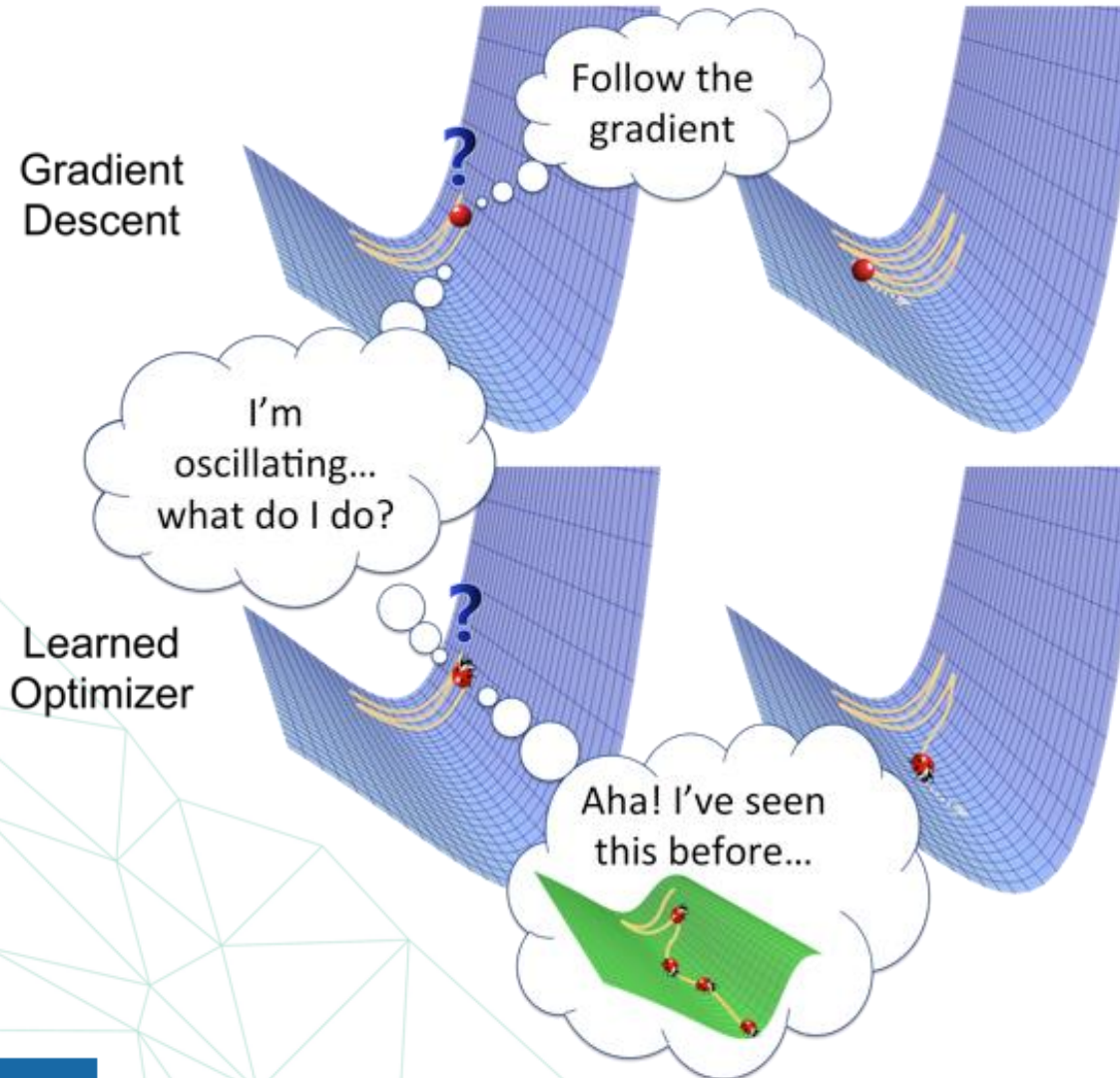
Optimization Techniques

Optimization Techniques

The goal of optimization is to minimize the loss function and improve the network's predictions. Gradient Descent is a widely used optimization algorithm, but several variants exist that can speed up and improve the training process.



Optimization Techniques



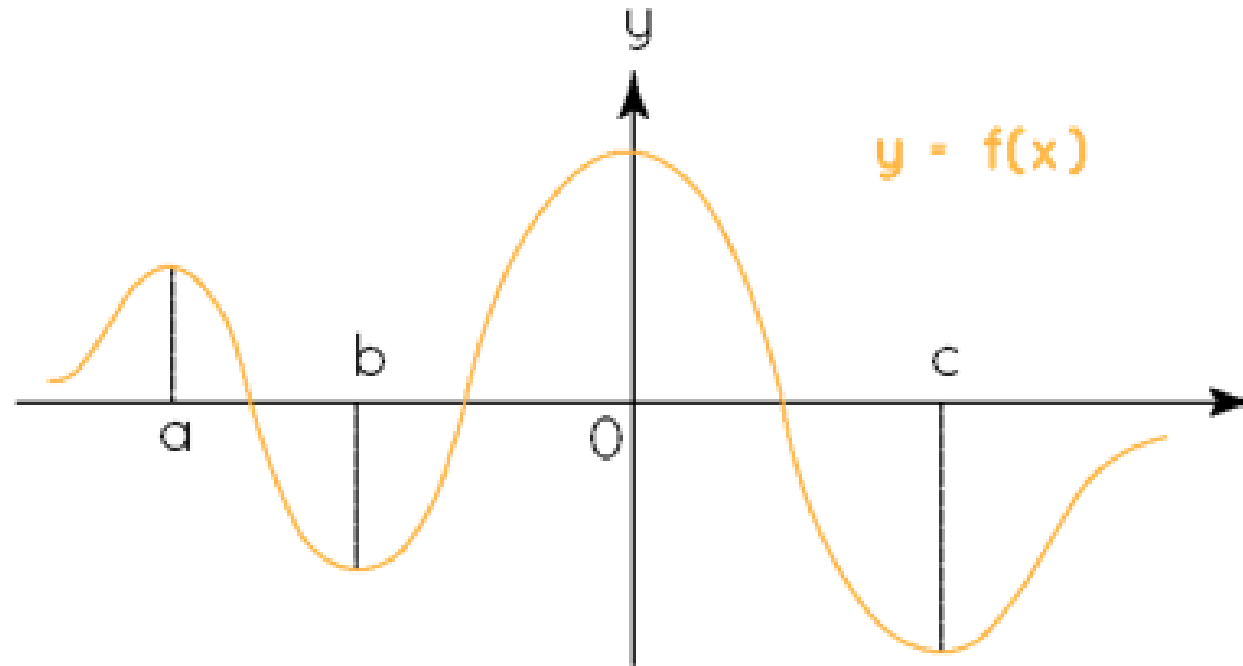
Optimization Techniques

Explanation of Maxima and Minima

- In mathematics and optimization problems, **Maxima** and **Minima** refer to the highest and lowest values that a function can take, respectively.
 - **Maxima**: The **largest** value a function reaches within a given domain or range.
 - **Minima**: The **smallest** value a function reaches within a given domain or range.

Optimization Techniques

Explanation of Maxima and Minima



Optimization Techniques

Global Maxima and Minima

- **Global Maximum:** The **highest point** of a function over the entire domain. No other value in the function is greater than this.
- **Global Minimum:** The **lowest point** of a function over the entire domain. No other value in the function is smaller than this.

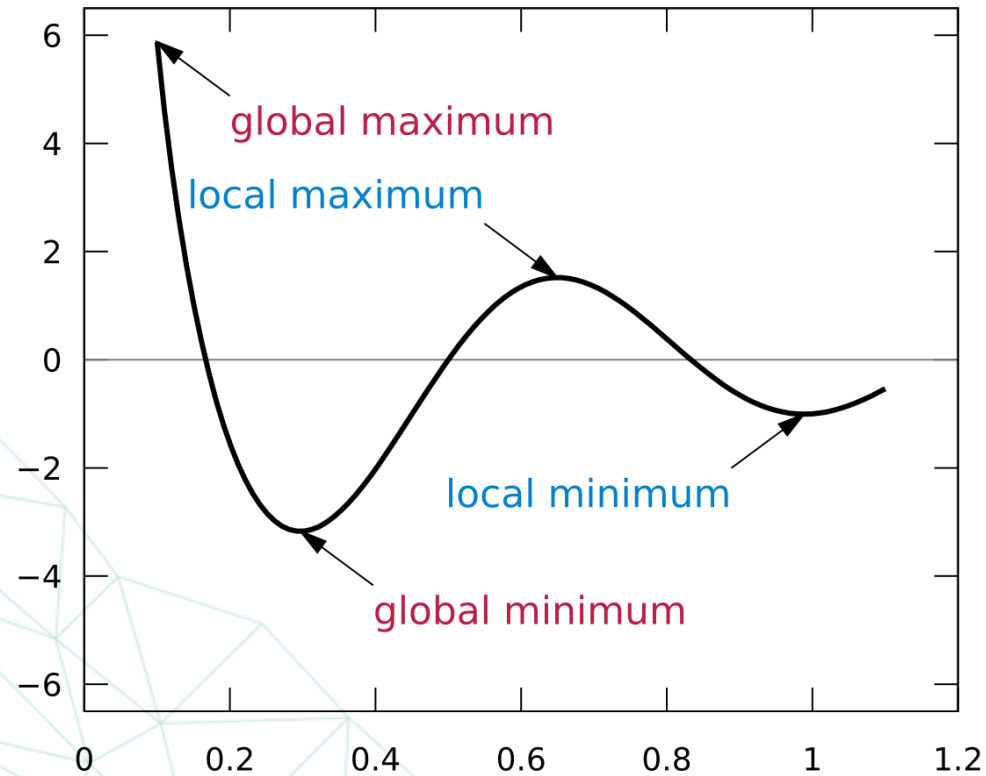
Optimization Techniques

Local Maxima and Minima

- **Local Maximum:** A point where the function reaches a peak within a specific region or range, but there may be higher points elsewhere in the function.
- **Local Minimum:** A point where the function reaches a trough within a specific region, but there may be lower points elsewhere.

Optimization Techniques

Global & Local Maxima and Minima



Optimization Techniques

Gradient Descent Variants

1. Batch Gradient Descent

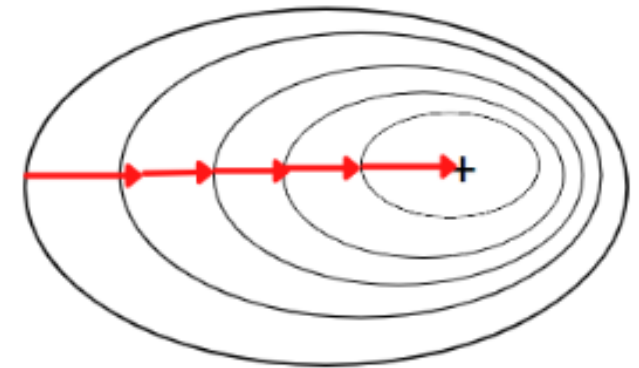
How it works ?

- The entire training dataset is used to compute the gradient and update the weights. This method is computationally expensive for large datasets.

Pros: Accurate gradient computation.

Cons: Slow and resource-intensive for large datasets.

Batch Gradient Descent



Optimization Techniques

Gradient Descent Variants

2. Stochastic Gradient Descent (SGD)

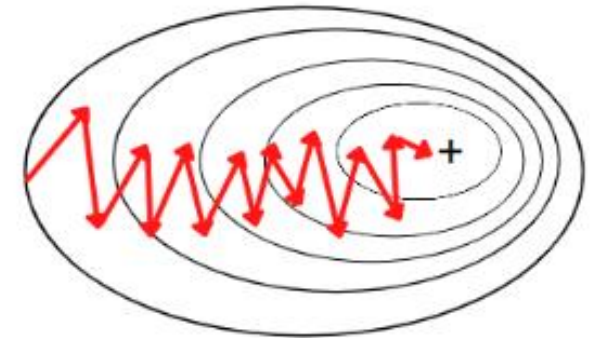
How it works ?

- SGD updates the weights after each training sample. Instead of using the entire dataset, it uses one sample at a time to compute the gradient and update the weights.

Pros: Faster than Batch Gradient Descent, and it introduces noise into the updates, which helps escape local minima.

Cons: The gradient estimates are noisy, and it can take longer to converge.

Stochastic Gradient Descent



Optimization Techniques

Gradient Descent Variants

3. Mini-batch Gradient Descent

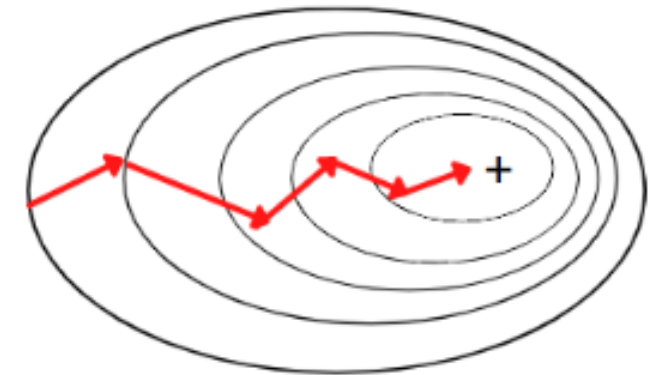
How it works ?

- Instead of using the entire dataset or a single sample, mini-batch gradient descent uses a small random subset (or batch) of data to compute the gradient and update the weights.

Pros: A balance between efficiency and speed, commonly used in practice.

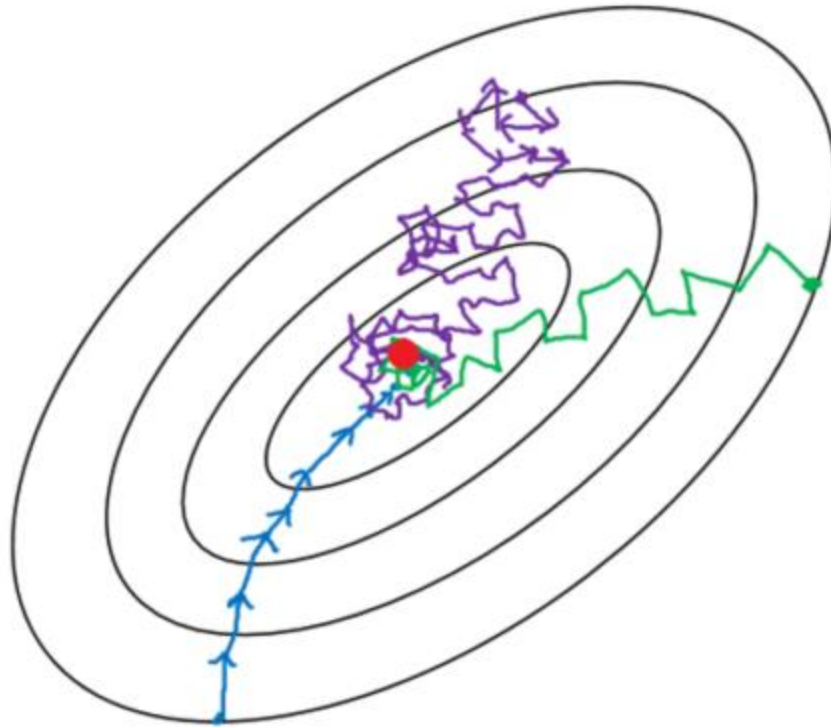
Cons: The batch size needs to be carefully chosen (too small = noisy, too large = slow).

Mini-Batch Gradient Descent



Optimization Techniques

Gradient Descent Variants



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

Optimization Techniques

Advanced Optimization Algorithms

1. Momentum

How it works ?

- Momentum helps accelerate gradient descent by adding a fraction of the previous update to the current update. This helps the algorithm converge faster, especially in regions with high curvature.

Why it helps ?

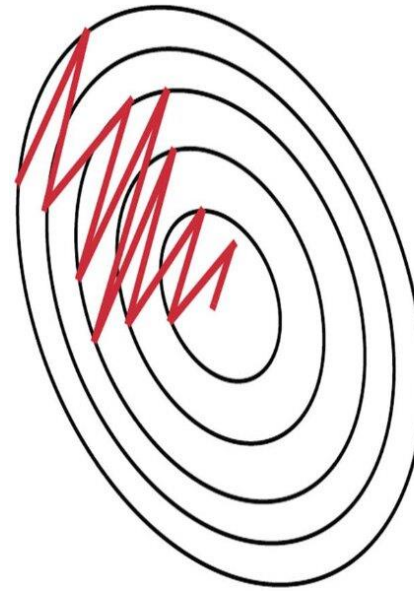
- Momentum helps smooth out the updates and prevents oscillations, leading to faster convergence

Optimization Techniques

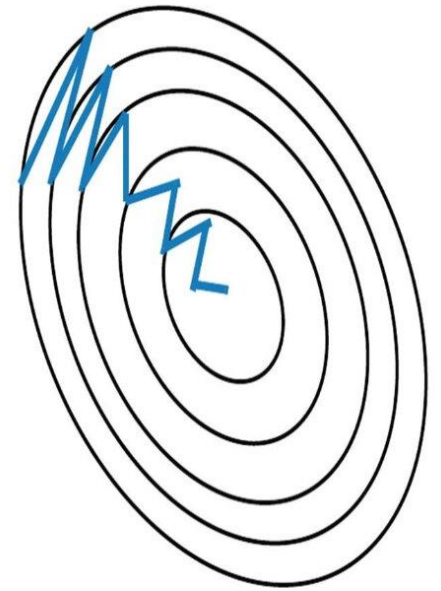
Advanced Optimization Algorithms

1. Momentum

- Momentum in gradient descent accelerates convergence by adding a speed parameter, reducing fluctuations, and smoothing the path toward the global minima, but it can overshoot if the learning rate is too high.



Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

Optimization Techniques

Advanced Optimization Algorithms

2. Adam (Adaptive Moment Estimation)

How it works ?

- Adam combines the advantages of both **RMSPprop** and **Momentum**. It uses an adaptive learning rate and maintains per-parameter learning rates based on moving averages of past gradients.

Why it helps ?

- Adam adapts the learning rate for each parameter individually, which allows for faster and more efficient convergence.

Optimization Techniques

Advanced Optimization Algorithms

2. Adam (Adaptive Moment Estimation)

Popular in Practice

- Adam is one of the most commonly used optimization algorithms due to its efficiency and effectiveness in training deep learning models.

Key Takeaways

1. **Deeper Neural Networks** allow the model to learn complex hierarchical features but come with challenges like the vanishing gradient problem, which can be solved using proper activation functions like ReLU.
2. **Overfitting** occurs when the network learns to memorize the training data instead of generalizing well. To prevent this, techniques like **L2 regularization**, **dropout**, and **early stopping** are used to regularize the network.
3. **Optimization Techniques** like **Adam** and **Momentum** are advanced methods to make gradient descent more efficient and help the model converge faster.